1. Problem Title & Link

Title: **LeetCode 992: Subarrays with K Different Integers**

Link: https://leetcode.com/problems/subarrays-with-k-different-integers/

2. Problem Statement (Short Summary)

You are given an integer array nums and an integer k.

Your task is to **count the number of subarrays that contain exactly k distinct integers**.

Important points:

- Subarrays must be **contiguous**
- We are **counting how many such subarrays exist**

3. Examples (Input → Output)

Example 1

Input

nums = [1,2,1,2,3]

k = 2

Output

7

Explanation

Valid subarrays with exactly 2 distinct integers:

[1,2]

[2,1]

[1,2]

[2,3]

[1,2,1]

[2,1,2]

[1,2,1,2]

Total = **7**

Example 2

Input

nums = [1,2,1,3,4]

k = 3

Output

3

Subarrays:

[1,2,1,3]

[2,1,3]

[1,3,4]

4. Constraints

1 ≤ nums.length ≤ 20,000

1 ≤ nums[i], k ≤ nums.length

Observations:

- Large input size
- Brute force $O(n^2)$ will be slow
- Need an **O(n)** or **O(n log n)** solution

5. Core Concept (Pattern / Topic)

Sliding Window + HashMap

Key idea:

Exactly K distinct =

At most K distinct

−

At most (K − 1) distinct

6. Thought Process (Step-by-Step Explanation)

Step 1 — Brute Force Idea

Check every subarray.

For each:

- Count distinct elements
- If distinct == k → increment answer

Time complexity:

$O(n^2)$

Too slow.

Step 2 — Key Insight

Instead of computing **exactly k**, we compute:

exactlyK = atMost(K) − atMost(K−1)

This converts the problem into two **simpler sliding window problems**.

Step 3 — Sliding Window for "At Most K"

Maintain:

left pointer

right pointer

frequency map

distinct count

Expand window with right.

If distinct count exceeds k, shrink window from left.

For each position:

subarrays += right − left + 1

7. Visual / Intuition Diagram (ASCII)

Example

nums = [1,2,1,2,3]

k = 2

Window expanding

[1]

[1,2]

[1,2,1]

[1,2,1,2]

Valid because distinct ≤ 2.

Add counts:

right - left + 1

Then shrink when distinct exceeds limit.

8. Pseudocode (Language Independent)

```
function atMost(k):

  left = 0
  count = 0
  map = {}

  for right in range(n):

    add nums[right] to map

    while map.size > k:

      remove nums[left]
      left++
```

```
        count += right - left + 1

    return count


answer = atMost(k) - atMost(k-1)
```

9. Code Implementation

Python

```
class Solution:
    def subarraysWithKDistinct(self, nums, k):

        def atMost(k):
            count = 0
            left = 0
            freq = {}

            for right in range(len(nums)):

                freq[nums[right]] = freq.get(nums[right],0) + 1

                while len(freq) > k:
                    freq[nums[left]] -= 1
                    if freq[nums[left]] == 0:
                        del freq[nums[left]]
                    left += 1

                count += right - left + 1

            return count

        return atMost(k) - atMost(k-1)
```

Java

```
class Solution {
```

```
    public int subarraysWithKDistinct(int[] nums, int k) {

        return atMost(nums, k) - atMost(nums, k-1);
    }

    private int atMost(int[] nums, int k){

        HashMap<Integer,Integer> map = new HashMap<>();

        int left = 0;
        int count = 0;

        for(int right = 0; right < nums.length; right++){

            map.put(nums[right], map.getOrDefault(nums[right],0) + 1);

            while(map.size() > k){

                map.put(nums[left], map.get(nums[left]) - 1);

                if(map.get(nums[left]) == 0)
                    map.remove(nums[left]);

                left++;
            }

            count += right - left + 1;
        }

        return count;
    }
}
```

10. Time & Space Complexity

Time Complexity

O(n)

Each element enters and leaves the window once.

Space Complexity

O(k)

HashMap stores up to k elements.

11. Common Mistakes / Edge Cases

1. Forgetting the formula

exactlyK = atMost(K) − atMost(K−1)

2. Incorrect map updates
3. Forgetting to remove element when frequency becomes zero
4. Infinite loop when shrinking window

12. Detailed Dry Run

Example

nums = [1,2,1,2,3]

k = 2

Compute:

atMost(2) = 12

atMost(1) = 5

Final answer

12 − 5 = 7

13. Common Use Cases (Real-Life / Interview)

Sliding window with frequency maps appears in:

- Log analysis
- Network packet monitoring
- Pattern detection in sequences
- Stream analytics

14. Common Traps (Important!)

1. Counting distinct elements incorrectly
2. Using brute force counting
3. Not shrinking window correctly
4. Forgetting difference formula

15. Builds To (Related LeetCode Problems)

Sliding Window progression

LC 209 Minimum Size Subarray Sum

LC 3 Longest Substring Without Repeating Characters

LC 424 Longest Repeating Character Replacement

LC 567 Permutation in String

LC 992 Subarrays with K Different Integers

16. Alternate Approaches + Comparison

Approach	Complexity	Notes
Brute Force	$O(n^2)$	too slow
Sliding Window + HashMap	$O(n)$	optimal

17. Why This Solution Works

Counting **exactly k distinct elements directly is difficult**.

But counting **at most k distinct elements** is easy with sliding window.

Subtracting two such counts isolates the **exactly k case**.

18. Variations / Follow-Up Questions

Possible interview variations:

1. Longest subarray with **k distinct elements**
2. Subarrays with **at most k distinct**
3. Subarrays with **exactly k even numbers**
4. Streaming version of the problem